



Prática - 2: Gestão de tarefas

Professor: Rogério Pereira Junior

rogerio.pereira@ifsc.edu.br

Laboratório de Sistemas Operacionais Atividade 1: Ciclo de Vida e Estados de Processos

Objetivo

Diferenciar o conceito estático (**Programa**) do conceito dinâmico (**tarefa/Tarefa**) e observar as transições de estados no Linux.

1. O Programa (Estático)

Um programa é apenas um arquivo no disco. Vamos localizar o binário do editor `nano`.

```
1 ls -l /usr/bin/nano
```

Observe que ele possui um tamanho em bytes e permissões, mas não consome CPU neste momento.

2. Criação e Suspensão (Mudança de Estado)

Vamos transformar o programa em um tarefa e suspendê-lo.

1. Execute o comando: `nano desafio.txt`
2. Dentro do editor, pressione as teclas **Ctrl + Z** ou **Ctrl + T** seguido de **Ctrl + Z**.

O sistema exibirá a mensagem "[1]+ Parado". O tarefa saiu do estado **Executando** para o estado **Espera/Suspensa**.

3. Investigando o tarefa Dinâmico

Use o comando abaixo para ver o "RG"(PID) e o estado (STAT) do tarefa:

```
1 ps -o pid,stat,comm | grep nano
```

O estado **T** indica que o tarefa está parado (Stopped).

4. Retorno à Execução

Traga o tarefa de volta para o primeiro plano:

```
1 fg
```

O tarefa volta para a fila de **Pronto** e entra em **Executando** imediatamente.

5. Finalização (Estado Terminado)

Saia do editor (**Ctrl + X**) e verifique a lista de tarefas novamente. *O tarefa não existe mais na RAM; ele foi movido para o estado **Terminado**.*

Resumo de Estados observados

- **Novo:** Quando digitamos o comando.
- **Executando:** O editor aberto na tela.
- **Espera (Suspendido):** Após o Ctrl + Z.
- **Pronto:** O breve instante após o comando fg.
- **Terminado:** Após fechar o programa.

Laboratório de Sistemas Operacionais

Atividade 2: Gestão de Tarefas

As práticas a seguir foram desenvolvidas para o sistema Ubuntu e têm como objetivo relacionar os conceitos teóricos de gestão de tarefas com sua implementação real no sistema operacional.

Prática 1 – Observando tarefas em execução

Objetivo: Compreender que cada programa em execução corresponde a uma tarefa (processo).

Comandos:

```
1 ps aux
2 top
3 htop
```

Atividades:

- Execute `ps aux` e identifique PID, usuário, CPU e memória.
- Execute `top` e observe a alternância de processos.

Conceitos associados: processos, multitarefa, escalonamento.

Questão: Por que a ordem dos processos muda constantemente no `top`?

Prática 2 – Múltiplas tarefas do mesmo programa

Objetivo: Mostrar que um mesmo programa pode gerar várias tarefas.

Comandos:

```
1 xeyes &
2 xeyes &
3 xeyes &
4 ps -C xeyes
```

Atividades:

- Observe que cada execução gera um PID diferente.

Conceitos associados: programa versus processo, instâncias de processos.

Prática 3 – Hierarquia de processos

Objetivo: Visualizar a relação pai–filho entre processos.

Comandos:

```
1 pstree
2 pstree -p
3 sleep 300
4 ps -ef
```

Atividades:

- Identifique o PID e o PPID do processo `sleep`.

Conceitos associados: hierarquia de processos, criação de processos.

Questão: O que acontece com um processo filho se o processo pai for encerrado?

Prática 4 – Estados dos processos

Objetivo: Identificar os estados do ciclo de vida das tarefas.

Comandos:

```
1 ps -o pid,ppid,stat,cmd
2 sleep 1000
3 Ctrl+Z
```

Atividades:

- Observe estados como R, S e T.

Conceitos associados: estados do processo, bloqueio e suspensão.

Prática 5 – Troca de contexto e concorrência

Objetivo: Observar a divisão da CPU entre tarefas concorrentes.

Comandos:

```
1 yes > /dev/null &
2 yes > /dev/null &
3 yes > /dev/null &
4 top
```

Atividades:

- Observe o uso da CPU e a alternância entre processos.

Conceitos associados: troca de contexto, tempo compartilhado.

Prática 6 – Prioridades e escalonamento

Objetivo: Analisar o impacto da prioridade no escalonamento.

Comandos:

```
1 nice -n 10 yes > /dev/null &
2 nice -n -5 yes > /dev/null &
3 top
```

Atividades:

- Compare o uso de CPU entre processos com prioridades diferentes.

Conceitos associados: prioridade, gestão de processos.

Prática 7 – Threads em execução

Objetivo: Visualizar múltiplas threads dentro de um processo.

Comandos:

```
1 ps -eLf
2 top -H
```

Atividades:

- Compare um processo simples (sleep) com uma aplicação gráfica.

Conceitos associados: threads, fluxos de execução.

Prática 8 – Finalização de tarefas

Objetivo: Encerrar processos utilizando sinais.

Comandos:

```
1 kill PID
2 kill -9 PID
3 pkill nome
```

Conceitos associados: sinais, controle de processos.

Questão: Qual a diferença entre kill e kill -9?

Prática 9 – Processo bloqueado e executando com xeyes

Objetivo: Visualizar processos aguardando eventos e processos executando.

Execução:

```
1 xeyes &
2 top
```

Atividades:

- Observe o processo em repouso (estado S).
- Mova o mouse e observe o aumento de CPU.

Conceitos associados: processo bloqueado, eventos, escalonamento.

Prática 10 – Suspensão e retomada de processos

Objetivo: Diferenciar processo bloqueado de processo suspenso.

Comandos:

```
1 Ctrl+Z
2 jobs
3 fg
4 ps -o pid,stat,cmd
```

Conceitos associados: suspensão, estado T (stopped).

Laboratório de Sistemas Operacionais

Atividade 3: Execução e finalização de processos

Abra um terminal linux e siga as instruções abaixo:

1. Liste a quantidade de memória total em uso informada pelo sistema operacional (dê uma olhada no comando `free`).
2. Liste todos os tarefas rodando no sistema
3. Digite o comando o comando **yes** » `/dev/null`
4. Abra outra aba do terminal (`Ctrl + Shift + t`) e analise usando `grep`, `ps aux` e `ps` o tarefa referente ao comando anterior
5. Quanto de CPU o tarefa do comando **yes** está consumindo?
6. Feche esta segunda aba e volte para o terminal anterior do comando **yes** e mate a tarefa
7. Mostrar no terminal linux os tarefas que estão consumindo mais processador, atualizar a listagem a cada 10 segundos.
8. Pesquisar pelo tarefa de nome "terminal" dentro da saída do comando **ps aux**. Use pipes
9. Pesquisar pelos tarefas do usuário **aluno** e guardar o resultado no arquivo **tarefas-aluno.txt**.
10. Abrir o arquivo **tarefas-aluno.txt** com o editor de texto pluma **em segundo plano**
11. Procure o ID do tarefa referente ao editor de texto e finalize o mesmo
12. Digite o comando **xeyes** (nota-se que é um programa com "olhos" que segue o movimento do mouse)
13. Suspenda o tarefa **via atalho**. O que aconteceu com o programa? Ele continuo seguindo o movimento do mouse?
14. Procure pelo ID do tarefa **xeyes** e mate-o através de seu ID.
15. Novamente digite o comando **xeyes**
16. Agora, em outro terminal suspenda o tarefa com o comando **kill**. Analise no terminal do tarefa o que aconteceu
17. Agora, retome o tarefa também com o comando **kill**. Analise o que aconteceu
18. Mate o tarefa no terminal do mesmo.
19. Digite novamente o comando **xeyes**
20. Suspenda o tarefa do **xeyes** via **atalho**
21. Agora retome o tarefa sem utilizar o comando **kill**. Lembre-se o tarefa esta **parado em segundo plano**
22. Qual a diferença entre os comandos `nice` e `renice`? Digite a resposta utilizando o editor nano em um arquivo **nice-renice.txt**
23. Crie um tarefa em segundo plano com o comando `yes` (enviando para `/dev/null`) com menor prioridade (PRI) possível
24. Abra o programa `xeyes` em segundo plano com prioridade (PRI) 25
25. Analise com o comando **htop** estas prioridades

26. Saia do **htop** e modifique esta prioridade do xeyes para 7.
27. Analise com o comando **htop** estas prioridades
28. Elimine estes dois últimos tarefas (xeyes e yes)
29. Pesquise o comando para mostrar informações sobre tarefas em execução em forma de árvore
30. Abra em segundo plano o firefox, dois xeyes, um editor pluma.
31. Em um segundo terminal execute o comando yes (enviando para /dev/null)
32. Abra um terceiro terminal e veja esses tarefas com o comando **htop**
33. Pelo **htop** pesquise pelo PID do tarefa do comando **yes**
34. Pelo **htop** mate o tarefa **yes**
35. Pelo **htop** filtre pelo PID do tarefa do comando **xeyes**
36. Pelo **htop** faça ordenação pelo PID
37. Pelo **htop** modifique a prioridade do tarefa do **firefox**
38. Pelo **htop** mate os tarefas restantes (2 xeyes, firefox e editor pluma)
39. Utilize o comando ps -eLf para visualizar as threads de cada processo. Observe como vários processos (como o navegador) possuem múltiplas threads rodando sob o mesmo PID.
40. Crie um pequeno programa em C que contenha um laço while(1) sem operações de entrada/saída
41. Compile e execute o programa. No top, observe o uso de CPU desse processo.
42. Verifique arquivos como status, maps (memória) e fd (arquivos abertos). Isso demonstra na prática o conteúdo de um PCB/TCB
43. Agora redirecione a saída do comando history para pratica2_nome.txt

Questionário de Entrega

1. Qual a diferença entre os estados **Ready** (Pronto) e **Running** (Executando) observada durante o uso do comando `top`?
2. Ao executar `ps tree`, você notou que o `xeyes` é filho de qual processo?
3. Explique por que, ao aumentarmos o valor de `nice` para 15, o processo tende a usar menos CPU quando há outros processos ativos.
4. O que acontece com o registro de um processo no SO quando enviamos um sinal `kill -9`?

Enviar no sigaa e questionário respondido bem como o arquivo `pratica2_nome.txt`